

LAPORAN PRAKTIKUM
POSTTEST(5)
ALGORITMA PEMROGRAMAN LANJUT



Disusun oleh:

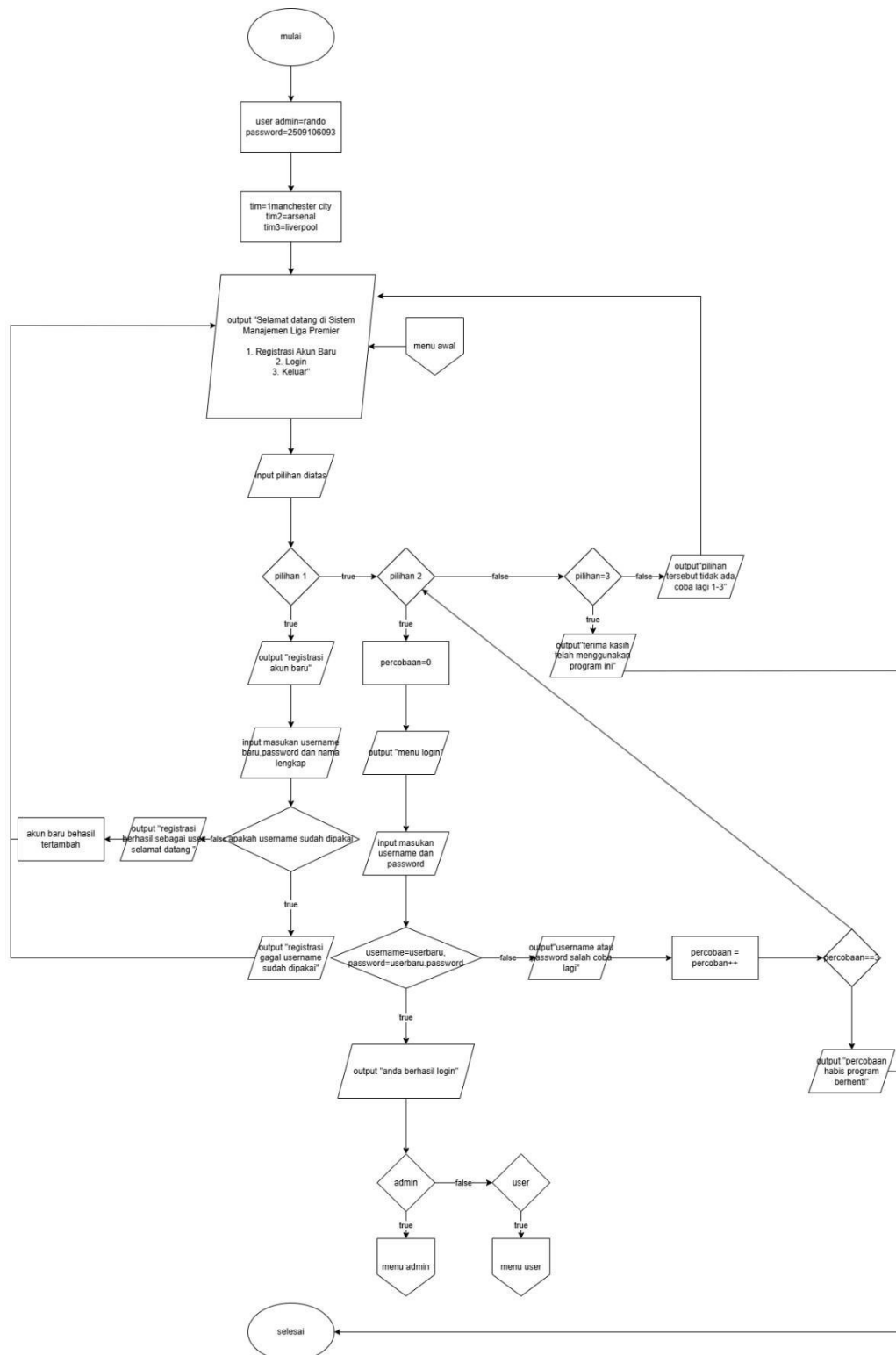
Nama: Isrando Mirhjah (2509106093)

Kelas (C"25)

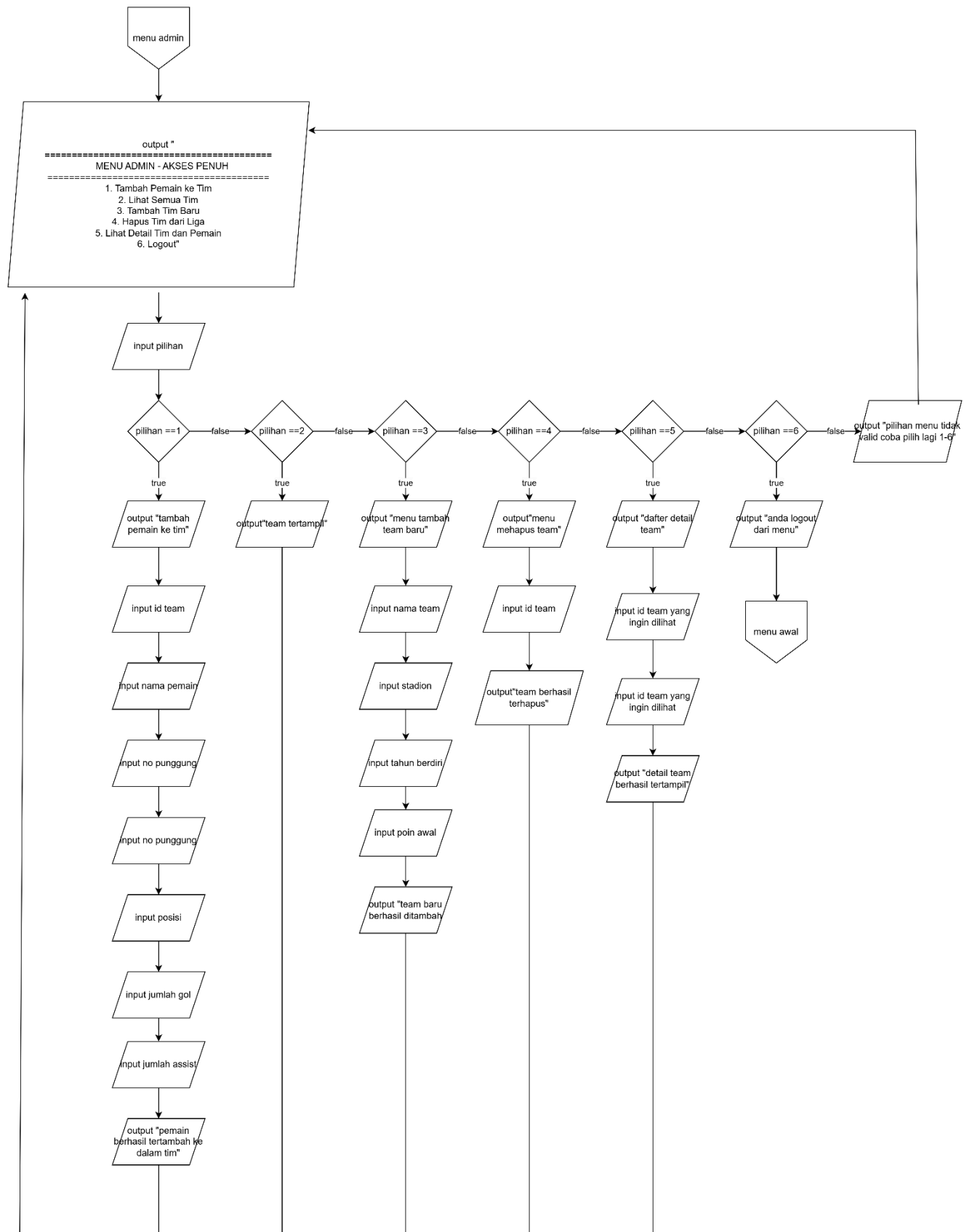
PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA

2025

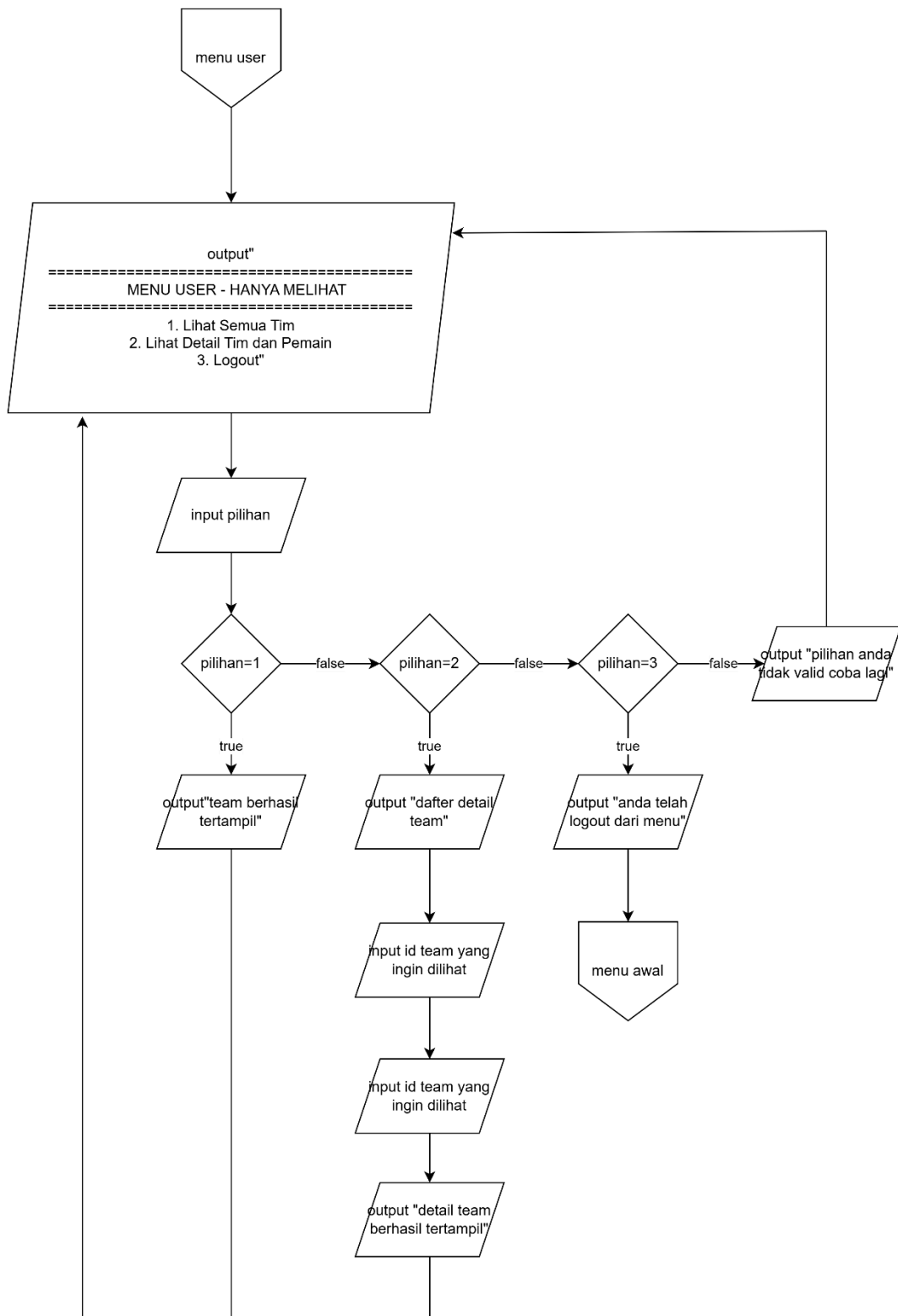
1. Flowchart



Flowchart 1.1 main menu



Flowchart 1.2 menu admin



Flowchart 1.3 menu user

Flowchart dia atas untuk menggambarkan cara alur program yang telah dibuat

2. Deskripsi Singkat Program

Program ini dibuat untuk agar kita bisa melihat detail team yang ada di premier league dan kita bisa menambahkan team baru dan menghapus team lama,kita juga bisa menambahkan pemain ke dalam team

3. Source Code

```
#include <iostream>
#include <iomanip>
#include <string>

using namespace std;

const size_t MAX_STREAM_SIZE = 1000;
const int INT_MIN_VALUE = -2147483648;
const int INT_MAX_VALUE = 2147483647;

void clearInputBuffer() {
    cin.ignore(MAX_STREAM_SIZE, '\n');
}

void clearScreenManual() {
    for (int i = 0; i < 50; i++) {
        cout << endl;
    }
}

int Integer(string pesan, int minValue, int maxValue) {
    int angka;
    bool valid = false;

    while (!valid) {
        cout << pesan;
        cin >> angka;

        if (cin.fail()) {
            cin.clear();
            clearInputBuffer();
            cout << "Input harus berupa angka! Silakan coba lagi.\n";
        } else if (angka < minValue || angka > maxValue) {
            cout << "Input harus antara " << minValue << " - " << maxValue
            << "! Silakan coba lagi.\n";
            clearInputBuffer();
        } else {
            clearInputBuffer();
            valid = true;
        }
    }
    return angka;
}

int Integer(string pesan) {
    return Integer(pesan, INT_MIN_VALUE, INT_MAX_VALUE);
}

struct Pemain {
    string namaPemain;
    int nomorPunggung;
    string posisi;
    int gol;
    int assist;
};
```

```

struct Tim {
    int id;
    string namaTim;
    string stadion;
    int tahunBerdiri;
    int poin;
    Pemain* skuad;
    int jumlahPemain;
    int kapasitasPemain;
};

struct User {
    string username;
    string password;
    string role;
    string namaLengkap;
};

struct UserNode {
    User data;
    UserNode* next;
};

class UserDatabase {
private:
    UserNode* head;
    int size;

public:
    UserDatabase() : head(nullptr), size(0) {}

    ~UserDatabase() {
        UserNode* current = head;
        while (current != nullptr) {
            UserNode* temp = current;
            current = current->next;
            delete temp;
        }
    }

    bool addUser(User user) {
        if (findUser(user.username) != nullptr) {
            return false;
        }

        UserNode* newNode = new UserNode;
        newNode->data = user;
        newNode->next = head;
        head = newNode;
        size++;
        return true;
    }

    User* findUser(string username) {
        UserNode* current = head;
        while (current != nullptr) {
            if (current->data.username == username) {
                return &(current->data);
            }
        }
    }
};

```

```

        }
        current = current->next;
    }
    return nullptr;
}

int getSize() { return size; }
};

const int MAKS_TIM = 20;
UserDatabase databaseUsers;

void inisialisasiTim(Tim* tim, int id, string nama, string stadion, int
tahun, int poin) {
    tim->id = id;
    tim->namaTim = nama;
    tim->stadion = stadion;
    tim->tahunBerdiri = tahun;
    tim->poin = poin;
    tim->jumlahPemain = 0;
    tim->kapasitasPemain = 11;
    tim->skواد = new Pemain[tim->kapasitasPemain];
}

void updatePoinTim(Tim& tim, int tambahanPoin) {
    tim.poin += tambahanPoin;
}

void tambahPoinTim(Tim* tim, int poin) {
    (*tim).poin += poin;
}

void updateStatistikPemain(Pemain* pemain, int golBaru, int assistBaru) {
    pemain->gol += golBaru;
    pemain->assist += assistBaru;
}

void tambahPemainKeTim(Tim* tim, Pemain* pemainBaru, bool* berhasil) {
    if (tim->jumlahPemain < tim->kapasitasPemain) {
        tim->skواد[tim->jumlahPemain] = *pemainBaru;
        tim->jumlahPemain++;
        *berhasil = true;
    } else {
        *berhasil = false;
    }
}

Pemain* cariPemainByNomor(Tim* tim, int nomorPunggung) {
    for (int i = 0; i < tim->jumlahPemain; i++) {
        if (tim->skواد[i].nomorPunggung == nomorPunggung) {
            return &(tim->skواد[i]);
        }
    }
    return nullptr;
}

int hitungTotalGol(Pemain* skواد, int indeks, int jumlahPemain) {
    if (indeks >= jumlahPemain) return 0;

```

```

        return skuad[indeks].gol + hitungTotalGol(skuad, indeks + 1,
jumlahPemain);
    }

    int hitungTotalAssist(Pemain* skuad, int indeks, int jumlahPemain) {
        if (indeks >= jumlahPemain) return 0;
        return skuad[indeks].assist + hitungTotalAssist(skuad, indeks + 1,
jumlahPemain);
    }

    void tampilkanPemain(Pemain* skuad, int indeks, int jumlahPemain) {
        if (indeks >= jumlahPemain) return;

        cout << left
            << setw(5) << indeks + 1
            << setw(25) << skuad[indeks].namaPemain.substr(0, 24)
            << setw(15) << skuad[indeks].nomorPunggung
            << setw(15) << skuad[indeks].posisi.substr(0, 14)
            << setw(10) << skuad[indeks].gol
            << setw(10) << skuad[indeks].assist
            << "\n";

        tampilkanPemain(skuad, indeks + 1, jumlahPemain);
    }

    void clearScreen() {
        clearScreenManual();
    }

    void tampilkanHeader(string judul) {
        cout << "\n===== \n";
        cout << "    " << judul << "\n";
        cout << "===== \n";
    }

    void tampilkanGaris(char ch = '=', int panjang = 50) {
        cout << string(panjang, ch) << endl;
    }

    void tampilkanPesan(string pesan, bool tungguEnter = true) {
        cout << pesan << endl;
        if (tungguEnter) {
            cout << "Tekan Enter untuk melanjutkan...";
            cin.get();
        }
    }

    int cariTim(int idTim, int jumlahTim, Tim daftarTim[]) {
        for (int i = 0; i < jumlahTim; i++) {
            if (daftarTim[i].id == idTim) return i;
        }
        return -1;
    }

    int cariTim(string namaTim, int jumlahTim, Tim daftarTim[]) {
        for (int i = 0; i < jumlahTim; i++) {
            if (daftarTim[i].namaTim == namaTim) return i;
        }
    }

```

```

        return -1;
    }

    void infoTim(Tim tim) {
        cout << "ID: " << tim.id << " | " << tim.namaTim
            << " (Pemain: " << tim.jumlahPemain << "/" << tim.kapasitasPemain
            << ", Poin: " << tim.poin << ")";
    }

    void infoTim(Tim tim, bool detailLengkap) {
        if (detailLengkap) {
            cout << "\n=== DETAIL TIM ===\n";
            cout << "ID Tim          : " << tim.id << "\n";
            cout << "Nama Tim           : " << tim.namaTim << "\n";
            cout << "Stadion            : " << tim.stadion << "\n";
            cout << "Tahun Berdiri      : " << tim.tahunBerdiri << "\n";
            cout << "Poin               : " << tim.poin << "\n";
            cout << "Jumlah Pemain      : " << tim.jumlahPemain << "/" <<
tim.kapasitasPemain << "\n";
        } else {
            infoTim(tim);
        }
    }

    bool validasiTahunBerdiri(int tahun) {
        return (tahun >= 1800 && tahun <= 2024);
    }

    bool validasiNomorPunggung(int nomor, Tim tim) {
        if (nomor < 1 || nomor > 99) return false;
        for (int i = 0; i < tim.jumlahPemain; i++) {
            if (tim.skvad[i].nomorPunggung == nomor) return false;
        }
        return true;
    }

    int inputAngka(string pesan, int minValue, int maxValue) {
        return Integer(pesan, minValue, maxValue);
    }

    int inputAngka(string pesan){
        return Integer(pesan);
    }

    bool registrasi(User* userBaru) {
        if (databaseUsers.findUser(userBaru->username) != nullptr) return false;
        if (userBaru->password.length() < 6) return false;

        userBaru->role = "User";
        return databaseUsers.addUser(*userBaru);
    }

    bool loginUser(string username, string password, User* currentUser) {
        User* user = databaseUsers.findUser(username);
        if (user != nullptr && user->password == password) {
            *currentUser = *user;
            return true;
        }
    }

```

```

        return false;
    }

bool tambahTimBaru(Tim* daftarTim, int* jumlahTim, int* nextId, Tim*
timBaru) {
    if (*jumlahTim >= MAKS_TIM) return false;

    timBaru->id = (*nextId)++;
    timBaru->jumlahPemain = 0;
    timBaru->kapasitasPemain = 11;
    timBaru->skواد = new Pemain[timBaru->kapasitasPemain];

    daftarTim[*jumlahTim] = *timBaru;
    (*jumlahTim)++;
    return true;
}

bool tambahPemainBaru(Tim* tim, Pemain* pemainBaru) {
    if (tim->jumlahPemain >= tim->kapasitasPemain) return false;
    tim->skواد[tim->jumlahPemain] = *pemainBaru;
    tim->jumlahPemain++;
    return true;
}

bool hapusTimDariLiga(Tim* daftarTim, int* jumlahTim, int idTim) {
    int indeks = cariTim(idTim, *jumlahTim, daftarTim);
    if (indeks == -1) return false;

    delete[] daftarTim[indeks].skواد;
    daftarTim[indeks].skواد = nullptr;

    for (int i = indeks; i < *jumlahTim - 1; i++) {
        daftarTim[i] = daftarTim[i + 1];
    }
    daftarTim[*jumlahTim - 1].skواد = nullptr;
    (*jumlahTim)--;
    return true;
}

void tampilkanStatistikTim(Tim* tim) {
    cout << "\n=== STATISTIK TIM: " << tim->namaTim << " ===\n";
    cout << "Total Gol Semua Pemain : " << hitungTotalGol(tim->skواد, 0,
tim->jumlahPemain) << endl;
    cout << "Total Assist Semua Pemain : " << hitungTotalAssist(tim->skواد,
0, tim->jumlahPemain) << endl;

    if (tim->jumlahPemain > 0) {
        Pemain* topScorer = &(tim->skواد[0]);
        for (int i = 1; i < tim->jumlahPemain; i++) {
            if (tim->skواد[i].gol > topScorer->gol) {
                topScorer = &(tim->skواد[i]);
            }
        }
        cout << "Top Scorer : " << topScorer->namaPemain
            << " (" << topScorer->gol << " gol)" << endl;
    }
}

```

```

void NamaAscending(Tim* daftarTim, int jumlahTim) {
    for (int i = 0; i < jumlahTim - 1; i++) {
        for (int j = 0; j < jumlahTim - i - 1; j++) {
            if (daftarTim[j].namaTim > daftarTim[j + 1].namaTim) {
                Tim temp = daftarTim[j];
                daftarTim[j] = daftarTim[j + 1];
                daftarTim[j + 1] = temp;
            }
        }
    }
}

void PoinDescending(Tim* daftarTim, int jumlahTim) {
    for (int i = 0; i < jumlahTim - 1; i++) {
        int maxIdx = i;
        for (int j = i + 1; j < jumlahTim; j++) {
            if (daftarTim[j].poin > daftarTim[maxIdx].poin) {
                maxIdx = j;
            }
        }
        if (maxIdx != i) {
            Tim temp = daftarTim[i];
            daftarTim[i] = daftarTim[maxIdx];
            daftarTim[maxIdx] = temp;
        }
    }
}

void TahunAscending(Tim* daftarTim, int jumlahTim) {
    for (int i = 1; i < jumlahTim; i++) {
        Tim key = daftarTim[i];
        int j = i - 1;
        while (j >= 0 && daftarTim[j].tahunBerdiri > key.tahunBerdiri) {
            daftarTim[j + 1] = daftarTim[j];
            j--;
        }
        daftarTim[j + 1] = key;
    }
}

void tampilkanTim(Tim* daftarTim, int jumlahTim, string metode) {
    clearScreen();
    tampilkanHeader("DAFTAR TIM - " + metode);

    if (jumlahTim == 0) {
        tampilkanPesan("Belum ada tim terdaftar.", true);
        return;
    }

    cout << left
         << setw(5) << "ID"
         << setw(25) << "Nama Tim"
         << setw(25) << "Stadion"
         << setw(15) << "Tahun"
         << setw(10) << "Poin"
         << setw(15) << "Pemain"
         << "\n";
    tampilkanGaris('-', 95);
}

```

```

    for (int i = 0; i < jumlahTim; i++) {
        cout << left
            << setw(5) << daftarTim[i].id
            << setw(25) << daftarTim[i].namaTim.substr(0, 24)
            << setw(25) << daftarTim[i].stadion.substr(0, 24)
            << setw(15) << daftarTim[i].tahunBerdiri
            << setw(10) << daftarTim[i].poin
            << setw(15) << daftarTim[i].jumlahPemain
            << "\n";
    }
    tampilkanGaris('-', 95);

    cout << "\nTekan Enter untuk kembali...";
    cin.get();
}

void registrasiMenu() {
    clearScreen();
    tampilkanHeader("REGISTRASI AKUN BARU");

    User userBaru;

    cout << "Masukkan Username: ";
    getline(cin, userBaru.username);
    cout << "Masukkan Password (minimal 6 karakter): ";
    getline(cin, userBaru.password);
    cout << "Masukkan Nama Lengkap: ";
    getline(cin, userBaru.namaLengkap);

    if (registrasi(&userBaru)) {
        cout << "\n===== \n";
        cout << "          REGISTRASI BERHASIL! \n";
        cout << "===== \n";
        cout << "Username      : " << userBaru.username << " \n";
        cout << "Nama          : " << userBaru.namaLengkap << " \n";
        cout << "Role          : User \n";
        cout << "===== \n";
        cout << "Silakan login menggunakan akun Anda. \n";
    } else {
        cout << "\n===== \n";
        cout << "          REGISTRASI GAGAL! \n";
        cout << "===== \n";
        if (databaseUsers.findUser(userBaru.username) != nullptr) {
            cout << "Username '" << userBaru.username << "' sudah
digunakan! \n";
        } else if (userBaru.password.length() < 6) {
            cout << "Password harus minimal 6 karakter! \n";
        } else {
            cout << "Terjadi kesalahan saat registrasi. \n";
        }
        cout << "===== \n";
    }
    cout << "\nTekan Enter untuk melanjutkan...";
    cin.get();
}

bool loginMenu(User* currentUser) {

```

```

string username, password;
int percobaan = 0;
const int maxPercobaan = 3;

while (percobaan < maxPercobaan) {
    clearScreen();
    tampilkanHeader("MENU LOGIN");
    cout << "Username: ";
    getline(cin, username);
    cout << "Password: ";
    getline(cin, password);

    if (loginUser(username, password, currentUser)) {
        tampilkanGaris('-');
        cout << "\nLOGIN BERHASIL!\n";
        cout << "Selamat datang, " << currentUser->namaLengkap << "!\n";
        cout << "Role: " << currentUser->role << "\n";
        tampilkanGaris('-');
        cout << "\nTekan Enter untuk melanjutkan...";
        cin.get();
        return true;
    }

    percobaan++;
    if (percobaan < maxPercobaan) {
        cout << "\nUsername atau password salah! Silakan coba lagi.\n";
        cout << "Sisa percobaan: " << (maxPercobaan - percobaan) <<
"\n";

        cout << "Tekan Enter untuk melanjutkan...";
        cin.get();
    }
}

clearScreen();
tampilkanHeader("LOGIN GAGAL");
cout << "\nAnda telah gagal login sebanyak " << maxPercobaan << "
kali.\n";
cout << "Tekan Enter untuk kembali ke menu utama...";
cin.get();
return false;
}

void lihatSemuaTim(Tim* daftarTim, int* jumlahTim) {
    clearScreen();
    tampilkanHeader("DAFTAR SEMUA TIM");

    if (*jumlahTim == 0) {
        tampilkanPesan("Belum ada tim terdaftar.", true);
        return;
    }

    cout << left
        << setw(5) << "ID"
        << setw(25) << "Nama Tim"
        << setw(25) << "Stadion"
        << setw(15) << "Tahun"
        << setw(10) << "Poin"
        << setw(15) << "Pemain"

```

```

        << "\n";
tampilkanGaris('-', 95);

for (int i = 0; i < *jumlahTim; i++) {
    cout << left
        << setw(5) << daftarTim[i].id
        << setw(25) << daftarTim[i].namaTim.substr(0, 24)
        << setw(25) << daftarTim[i].stadion.substr(0, 24)
        << setw(15) << daftarTim[i].tahunBerdiri
        << setw(10) << daftarTim[i].poin
        << setw(15) << daftarTim[i].jumlahPemain
        << "\n";
}
tampilkanGaris('=', 95);

cout << "\nTekan Enter untuk kembali...";
cin.get();
}

void lihatDetailTim(Tim* daftarTim, int* jumlahTim, int* nextIdTim) {
    clearScreen();
    tampilkanHeader("DETAIL TIM DAN PEMAIN");

    if (*jumlahTim == 0) {
        tampilkanPesan("Belum ada tim terdaftar.", true);
        return;
    }

    lihatSemuaTim(daftarTim, jumlahTim);

    int idTim = inputAngka("\nMasukkan ID Tim yang ingin dilihat detailnya:
", 1, *nextIdTim - 1);
    int indeksTim = cariTim(idTim, *jumlahTim, daftarTim);

    if (indeksTim == -1) {
        tampilkanPesan("Tim dengan ID " + to_string(idTim) + " tidak
ditemukan!", true);
        return;
    }

    clearScreen();
    infoTim(daftarTim[indeksTim], true);

    if (daftarTim[indeksTim].jumlahPemain == 0) {
        cout << "\nBelum ada pemain di tim ini.\n";
    } else {
        cout << "\nDAFTAR PEMAIN:\n";
        tampilkanGaris('-', 80);
        cout << left
            << setw(5) << "No"
            << setw(25) << "Nama Pemain"
            << setw(15) << "No Punggung"
            << setw(15) << "Posisi"
            << setw(10) << "Gol"
            << setw(10) << "Assist"
            << "\n";
        tampilkanGaris('-', 80);
        tampilkanPemain(daftarTim[indeksTim].skuat, 0,

```

```

daftarTim[indeksTim].jumlahPemain);
    tampilkanGaris('-', 80);
    tampilkanStatistikTim(&daftarTim[indeksTim]);
}

cout << "\nTekan Enter untuk kembali...";
cin.get();
}

void tambahTimMenu(Tim* daftarTim, int* jumlahTim, int* nextIdTim) {
    clearScreen();
    tampilkanHeader("TAMBAH TIM BARU");

    if (*jumlahTim >= MAKS_TIM) {
        tampilkanPesan("Database tim sudah penuh (maksimal " +
to_string(MAKS_TIM) + " tim)!", true);
        return;
    }

    Tim timBaru;
    timBaru.skuad = nullptr;

    cout << "Nama Tim: ";
    getline(cin, timBaru.namaTim);
    cout << "Stadion: ";
    getline(cin, timBaru.stadion);

    do {
        timBaru.tahunBerdiri = inputAngka("Tahun Berdiri (1800-2024): ",
1800, 2024);
    } while (!validasiTahunBerdiri(timBaru.tahunBerdiri));

    timBaru.poin = inputAngka("Poin Awal (0-100): ", 0, 100);

    if (tambahTimBaru(daftarTim, jumlahTim, nextIdTim, &timBaru)) {
        tampilkanPesan("\nTim " + timBaru.namaTim + " berhasil ditambahkan!
ID Tim: " +
to_string(daftarTim[*jumlahTim - 1].id), true);
    } else {
        tampilkanPesan("\nGagal menambahkan tim!", true);
    }
}

void tambahPemainMenu(Tim* daftarTim, int* jumlahTim, int* nextIdTim) {
    clearScreen();
    tampilkanHeader("TAMBAH PEMAIN KE TIM");

    if (*jumlahTim == 0) {
        tampilkanPesan("Belum ada tim terdaftar. Tambah tim terlebih
dahulu.", true);
        return;
    }

    cout << "DAFTAR TIM:\n";
    tampilkanGaris('-', 50);
    for (int i = 0; i < *jumlahTim; i++) {
        infoTim(daftarTim[i]);
        cout << endl;
    }
}

```

```

    }
    tampilkanGaris('-', 50);

    int idTim = inputAngka("\nMasukkan ID Tim: ", 1, *nextIdTim - 1);
    int indeksTim = cariTim(idTim, *jumlahTim, daftarTim);

    if (indeksTim == -1) {
        tampilkanPesan("Tim dengan ID " + to_string(idTim) + " tidak
ditemukan!", true);
        return;
    }

    if (daftarTim[indeksTim].jumlahPemain >=
daftarTim[indeksTim].kapasitasPemain) {
        tampilkanPesan("Tim sudah mencapai kapasitas maksimal (" +
to_string(daftarTim[indeksTim].kapasitasPemain) + "
pemain)!", true);
        return;
    }

    cout << "\nMenambah pemain untuk tim: " << daftarTim[indeksTim].namaTim
<< "\n";
    cout << "Jumlah pemain saat ini: " << daftarTim[indeksTim].jumlahPemain
<< "/" << daftarTim[indeksTim].kapasitasPemain << "\n\n";

    Pemain pemainBaru;
    cout << "Nama Pemain: ";
    getline(cin, pemainBaru.namaPemain);

    do {
        pemainBaru.nomorPunggung = inputAngka("Nomor Punggung (1-99): ", 1,
99);
        if (!validasiNomorPunggung(pemainBaru.nomorPunggung,
daftarTim[indeksTim])) {
            cout << "Nomor punggung sudah digunakan atau tidak valid!\n";
        }
    } while (!validasiNomorPunggung(pemainBaru.nomorPunggung,
daftarTim[indeksTim]));

    cout << "Posisi: ";
    getline(cin, pemainBaru.posisi);
    pemainBaru.gol = inputAngka("Jumlah Gol: ", 0, 999);
    pemainBaru.assist = inputAngka("Jumlah Assist: ", 0, 999);

    bool berhasil = false;
    tambahPemainKeTim(&daftarTim[indeksTim], &pemainBaru, &berhasil);

    if (berhasil) {
        tampilkanPesan("\nPemain " + pemainBaru.namaPemain + " berhasil
ditambahkan ke " +
                        daftarTim[indeksTim].namaTim + "!", true);
    } else {
        tampilkanPesan("\nGagal menambahkan pemain!", true);
    }
}

void hapusTimMenu(Tim* daftarTim, int* jumlahTim, int* nextIdTim) {
    clearScreen();

```

```

    tampilkanHeader("HAPUS TIM DARI LIGA");

    if (*jumlahTim == 0) {
        tampilkanPesan("Belum ada tim terdaftar.", true);
        return;
    }

    lihatSemuaTim(daftarTim, jumlahTim);

    int idTim = inputAngka("\nMasukkan ID Tim yang akan dihapus: ", 1,
        *nextIdTim - 1);
    int indeksHapus = cariTim(idTim, *jumlahTim, daftarTim);

    if (indeksHapus == -1) {
        tampilkanPesan("\nTim dengan ID " + to_string(idTim) + " tidak
ditemukan!", true);
        return;
    }

    cout << "\nAnda akan menghapus tim:\n";
    cout << "Nama Tim      : " << daftarTim[indeksHapus].namaTim << "\n";
    cout << "Stadion        : " << daftarTim[indeksHapus].stadion << "\n";
    cout << "Jumlah Pemain : " << daftarTim[indeksHapus].jumlahPemain << "
pemain\n\n";
    cout << "Yakin ingin menghapus tim ini? (y/n): ";

    char konfirmasi;
    cin >> konfirmasi;
    clearInputBuffer();

    if (konfirmasi == 'y' || konfirmasi == 'Y') {
        if (hapusTimDariLiga(daftarTim, jumlahTim, idTim)) {
            tampilkanPesan("\nTim berhasil dihapus dari liga!", true);
        } else {
            tampilkanPesan("\nGagal menghapus tim!", true);
        }
    } else {
        tampilkanPesan("\nPenghapusan dibatalkan.", true);
    }
}

void menuAdmin(User* currentUser, Tim* daftarTim, int* jumlahTim, int*
nextIdTim) {
    int pilihan;
    bool running = true;
    Tim timTemp[MAKS_TIM];

    do {
        clearScreen();
        tampilkanHeader("MENU ADMIN");
        cout << "User: " << currentUser->username << " | " << currentUser->
namaLengkap << "\n\n";
        cout << "1. Tambah Pemain ke Tim\n";
        cout << "2. Lihat Semua Tim\n";
        cout << "3. Tambah Tim Baru\n";
        cout << "4. Hapus Tim dari Liga\n";
        cout << "5. Lihat Detail Tim dan Pemain\n";
        cout << "6. melihat tim\n";
    } while (running);
}

```

```

        cout << "7. melihat poin tim\n";
        cout << "8. tahun berdiri tim\n";
        cout << "9. Logout\n\n";

        pilihan = inputAngka("Pilih menu (1-9): ", 1, 9);

        switch (pilihan) {
            case 1: tambahPemainMenu(daftarTim, jumlahTim,
nextIdTim); break;
            case 2: lihatSemuaTim(daftarTim,
jumlahTim); break;
            case 3: tambahTimMenu(daftarTim, jumlahTim,
nextIdTim); break;
            case 4: hapusTimMenu(daftarTim, jumlahTim,
nextIdTim); break;
            case 5: lihatDetailTim(daftarTim, jumlahTim,
nextIdTim); break;
            case 6:
                for (int i = 0; i < *jumlahTim; i++) timTemp[i] =
daftarTim[i];
                NamaAscending(timTemp, *jumlahTim);
                tampilkanTim(timTemp, *jumlahTim, "URUTAN NAMA (A-Z)");
                break;
            case 7:
                for (int i = 0; i < *jumlahTim; i++) timTemp[i] =
daftarTim[i];
                PoinDescending(timTemp, *jumlahTim);
                tampilkanTim(timTemp, *jumlahTim, "URUTAN POIN (TERTINGGI ->
TERENDAH)");
                break;
            case 8:
                for (int i = 0; i < *jumlahTim; i++) timTemp[i] =
daftarTim[i];
                TahunAscending(timTemp, *jumlahTim);
                tampilkanTim(timTemp, *jumlahTim, "URUTAN TAHUN BERDIRI
(TERTUA -> TERMUDA)");
                break;
            case 9:
                cout << "\nLogout berhasil!\n";
                running = false;
                break;
        }
    } while (running);
}

void menuUser(User* currentUser, Tim* daftarTim, int* jumlahTim, int*
nextIdTim) {
    int pilihan;
    bool running = true;
    Tim timTemp[MAKS_TIM];

    do {
        clearScreen();
        tampilkanHeader("MENU USER");
        cout << "User: " << currentUser->username << " | " << currentUser->
namaLengkap << "\n\n";
        cout << "1. Lihat Semua Tim\n";
        cout << "2. Lihat Detail Tim dan Pemain\n";
    } while (running);
}

```

```

        cout << "3. melihat tim\n";
        cout << "4. melihat poin Tim \n";
        cout << "5. tahun berdiri Tim \n";
        cout << "6. Logout\n\n";

        pilihan = inputAngka("Pilih menu (1-6): ", 1, 6);

        switch (pilihan) {
            case 1: lihatSemuaTim(daftarTim, jumlahTim); break;
            case 2: lihatDetailTim(daftarTim, jumlahTim, nextIdTim); break;
            case 3:
                for (int i = 0; i < *jumlahTim; i++) timTemp[i] =
daftarTim[i];
                NamaAscending(timTemp, *jumlahTim);
                tampilkanTim(timTemp, *jumlahTim, "URUTAN NAMA (A-Z)");
                break;
            case 4:
                for (int i = 0; i < *jumlahTim; i++) timTemp[i] =
daftarTim[i];
                PoinDescending(timTemp, *jumlahTim);
                tampilkanTim(timTemp, *jumlahTim, "URUTAN POIN (TERTINGGI ->
TERENDAH)");
                break;
            case 5:
                for (int i = 0; i < *jumlahTim; i++) timTemp[i] =
daftarTim[i];
                TahunAscending(timTemp, *jumlahTim);
                tampilkanTim(timTemp, *jumlahTim, "URUTAN TAHUN BERDIRI
(TERTUA -> TERMUDA)");
                break;
            case 6:
                cout << "\nLogout berhasil!\n";
                running = false;
                break;
        }
    } while (running);
}

void inisialisasiData(Tim* daftarTim, int* jumlahTim, int* nextIdTim) {
    User admin;
    admin.username = "rando";
    admin.password = "2509106093";
    admin.namaLengkap = "Administrator";
    admin.role = "Admin";
    databaseUsers.addUser(admin);

    User user;
    user.username = "rando1";
    user.password = "093";
    user.namaLengkap = "User";
    user.role = "User";
    databaseUsers.addUser(user);

    if (*jumlahTim == 0) {
        inisialisasiTim(&daftarTim[*jumlahTim], (*nextIdTim)++,
                        "Manchester City", "Etihad Stadium", 1880, 73);
        (*jumlahTim)++;
    }
}

```

```

        inisialisasiTim(&daftarTim[*jumlahTim], (*nextIdTim)++,
                        "Arsenal", "Emirates Stadium", 1886, 71);
        (*jumlahTim)++;

        inisialisasiTim(&daftarTim[*jumlahTim], (*nextIdTim)++,
                        "Liverpool", "Anfield", 1892, 70);
        (*jumlahTim)++;
    }
}

void cleanupData(Tim* daftarTim, int* jumlahTim) {
    for (int i = 0; i < *jumlahTim; i++) {
        delete[] daftarTim[i].skuad;
        daftarTim[i].skuad = nullptr;
    }
    *jumlahTim = 0;
}

int main() {
    Tim daftarTim[MAKS_TIM];
    int jumlahTim = 0;
    int nextIdTim = 1;

    for (int i = 0; i < MAKS_TIM; i++) {
        daftarTim[i].skuad = nullptr;
    }

    inisialisasiData(daftarTim, &jumlahTim, &nextIdTim);

    int pilihan;
    bool appRunning = true;

    while (appRunning) {
        clearScreen();
        tampilkanHeader("SISTEM MANAJEMEN LIGA SEPAK BOLA");
        cout << "1. Login\n";
        cout << "2. Registrasi\n";
        cout << "3. Keluar\n\n";

        pilihan = inputAngka("Pilih menu (1-3): ", 1, 3);

        switch (pilihan) {
            case 1: {
                User currentUser;
                if (loginMenu(&currentUser)) {
                    if (currentUser.role == "Admin") {
                        menuAdmin(&currentUser, daftarTim, &jumlahTim,
&nextIdTim);
                    } else {
                        menuUser(&currentUser, daftarTim, &jumlahTim,
&nextIdTim);
                    }
                }
                break;
            }
            case 2:
                registrasiMenu();
                break;
        }
    }
}

```

```
        case 3:
            cout << "\nTerima kasih! Sampai jumpa.\n";
            appRunning = false;
            break;
    }
}

cleanupData(daftarTim, &jumlahTim);
return 0;
}
```

4. Hasil Output

```
=====
      SISTEM MANAJEMEN LIGA PREMIER
=====
Selamat datang di Sistem Manajemen Liga Premier

1. Registrasi Akun Baru
2. Login
3. Keluar

Pilih menu (1-3):
```

Gambar 4.1 output menu awal

```
=====
      REGISTRASI AKUN BARU
=====
Masukkan Username: randooioi
Masukkan Password (minimal 6 karakter): randooioio
Masukkan Nama Lengkap: rando

=====
      REGISTRASI BERHASIL!
=====
Username      : randooioi
Nama          : rando
Role          : User
=====
Silakan login menggunakan akun Anda.

Tekan Enter untuk melanjutkan...█
```

Gambar 4.2 output registrasi akun sebagai user

```
=====
  MENU LOGIN
=====
Username: rando123
Password: rando123
-----

LOGIN BERHASIL!
Selamat datang, rando!
Role: Admin
-----

Tekan Enter untuk melanjutkan...█
```

Gambar 4.3 output login menu berhasil

```
=====
  LOGIN GAGAL
=====

Anda telah gagal login 3 kali.
Tekan Enter untuk keluar...
PS D:\praktikum-apl\post-test\post-test-2> █
```

Gambar 4.4 output ini ketika kita salah password hingga 3 kali

```
=====
MENU ADMIN
=====
User: rando | Administrator

1. Tambah Pemain ke Tim
2. Lihat Semua Tim
3. Tambah Tim Baru
4. Hapus Tim dari Liga
5. Lihat Detail Tim dan Pemain
6. melihat tim
7. melihat poin tim
8. tahun berdiri tim
9. Logout

Pilih menu (1-9): █
```

Gambar 4.5 output menu admin

```
=====
TAMBAH PEMAIN KE TIM
=====
DAFTAR TIM:
-----
ID: 1 | Manchester City (Pemain: 0/11)
ID: 2 | Arsenal (Pemain: 0/11)
ID: 3 | Liverpool (Pemain: 0/11)
-----

Masukkan ID Tim: 2

Menambah pemain untuk tim: Arsenal
Jumlah pemain saat ini: 0/11

Nama Pemain: rando
Nomor Punggung: 10
Posisi: st
Jumlah Gol: 9
Jumlah Assist: 10

Pemain rando berhasil ditambahkan ke Arsenal!
Tekan Enter untuk kembali...█
```

Gambar 4.6 output untuk menambahkan pemain ke team

```
=====
DAFTAR SEMUA TIM
=====
```

ID	Nama Tim	Stadion	Tahun Berdiri	Poin	Jumlah Pemain
1	Manchester City	Etihad Stadium	1880	73	0
2	Arsenal	Emirates Stadium	1886	71	1
3	Liverpool	Anfield	1892	70	0

```
=====
Tekan Enter untuk kembali...█
```

Gambar 4.7 output untuk melihat semua team dan poin

```
=====
TAMBAH TIM BARU
=====
Nama Tim: barcelona
Stadion: segiri
Tahun Berdiri: 1945
Poin Awal: 89

Tim barcelona berhasil ditambahkan!
ID Tim: 4
Tekan Enter untuk kembali...█
```

Gambar 4.8 output untuk menambahkan team baru ke liga

```
=====
HAPUS TIM DARI LIGA
=====
DAFTAR TIM:
-----
ID    Nama Tim           Stadion              Poin    Jumlah Pemain
-----
1     Manchester City      Etihad Stadium      73      0
2     Arsenal               Emirates Stadium    71      1
3     Liverpool              Anfield             70      0
4     barcelona              segiri              89      0
=====

Masukkan ID Tim yang akan dihapus: 1

Anda akan menghapus tim:
Nama Tim      : Manchester City
Stadion       : Etihad Stadium
Jumlah Pemain : 0 pemain

Yakin ingin menghapus tim ini? (y/n): y

Tim Manchester City berhasil dihapus dari liga!

Tekan Enter untuk kembali... 
```

Gambar 4.9 output untuk menghapus team

```
=====
DETAIL TIM
=====
ID Tim      : 2
Nama Tim    : Arsenal
Stadion     : Emirates Stadium
Tahun Berdiri : 1886
Poin        : 71
Jumlah Pemain : 1/11

DAFTAR PEMAIN:
-----
No  Nama Pemain           No Punggung  Posisi    Gol    Assist
-----
1   rando                 10           st         9      10
=====

Tekan Enter untuk kembali... 
```

Gambar 5.0 output ingin melihat detail team

```
=====
  DAFTAR TIM - URUTAN NAMA (A-Z)
=====
```

ID	Nama Tim	Stadion	Tahun	Poin	Pemain
2	Arsenal	Emirates Stadium	1886	71	0
3	Liverpool	Anfield	1892	70	0
1	Manchester City	Etihad Stadium	1880	73	0

```
=====
```

Gambar 5.1 output melihat tim secara ascending

```
=====
  DAFTAR TIM - URUTAN POIN (TERTINGGI -> TERENDAH)
=====
```

ID	Nama Tim	Stadion	Tahun	Poin	Pemain
1	Manchester City	Etihad Stadium	1880	73	0
2	Arsenal	Emirates Stadium	1886	71	0
3	Liverpool	Anfield	1892	70	0

```
=====
```

Gambar 5.2 output melihat poin tim secara descending

```
=====
  DAFTAR TIM - URUTAN TAHUN BERDIRI (TERTUA -> TERMUDA)
=====
```

ID	Nama Tim	Stadion	Tahun	Poin	Pemain
1	Manchester City	Etihad Stadium	1880	73	0
2	Arsenal	Emirates Stadium	1886	71	0
3	Liverpool	Anfield	1892	70	0

```
=====
```

Gambar 5.3 output melihat tahun berdiri tim secara ascending

```
=====
MENU USER
=====
User: rando1 | User

1. Lihat Semua Tim
2. Lihat Detail Tim dan Pemain
3. melihat tim
4. melihat poin Tim
5. tahun berdiri Tim
6. Logout

Pilih menu (1-6): █
```

Gambar 5.4 output menu user

```
=====
DAFTAR SEMUA TIM
=====
ID  Nama Tim           Stadion              Tahun Berdiri  Poin  Jumlah Pemain
-----
2   Arsenal            Emirates Stadium    1886           71    1
3   Liverpool          Anfield            1892           70    0
4   barcelona          segiri             1945           89    0
=====
Tekan Enter untuk kembali...█
```

Gambar 5.5 output untuk melihat semua team dan poin

```
=====
DETAIL TIM
=====
ID Tim      : 2
Nama Tim    : Arsenal
Stadion     : Emirates Stadium
Tahun Berdiri : 1886
Poin        : 71
Jumlah Pemain : 1/11

DAFTAR PEMAIN:
-----
No  Nama Pemain           No Punggung  Posisi  Gol  Assist
-----
1   rando                  10           st      9    10
=====
Tekan Enter untuk kembali...█
```

Gambar 5.6 output ingin melihat detail team

```
=====
  DAFTAR TIM - URUTAN NAMA (A-Z)
=====
```

ID	Nama Tim	Stadion	Tahun	Poin	Pemain
2	Arsenal	Emirates Stadium	1886	71	0
3	Liverpool	Anfield	1892	70	0
1	Manchester City	Etihad Stadium	1880	73	0

```
=====
```

Gambar 5.7 output melihat tim secara ascending

```
=====
  DAFTAR TIM - URUTAN POIN (TERTINGGI -> TERENDAH)
=====
```

ID	Nama Tim	Stadion	Tahun	Poin	Pemain
1	Manchester City	Etihad Stadium	1880	73	0
2	Arsenal	Emirates Stadium	1886	71	0
3	Liverpool	Anfield	1892	70	0

```
=====
```

Gambar 5.8 output melihat poin tim secara descending

```
=====
  DAFTAR TIM - URUTAN TAHUN BERDIRI (TERTUA -> TERMUDA)
=====
```

ID	Nama Tim	Stadion	Tahun	Poin	Pemain
1	Manchester City	Etihad Stadium	1880	73	0
2	Arsenal	Emirates Stadium	1886	71	0
3	Liverpool	Anfield	1892	70	0

```
=====
```

Gambar 5.9 output melihat tahun berdiri tim secara ascending

```
=====
      SISTEM MANAJEMEN LIGA PREMIER
=====
Selamat datang di Sistem Manajemen Liga Premier

1. Registrasi Akun Baru
2. Login
3. Keluar

Pilih menu (1-3): 3

Terima kasih telah menggunakan program ini!
PS D:\praktikum-apl\post-test\post-test-2> |
```

Gambar 6.0 output untuk keluar dari program

5. Langkah-langkah GIT

5.1 GIT Add

```
PS D:\praktikum-apl\post-test> git add .
```

Gambar 5.1 git add itu dipakai untuk menandai file yang berubah supaya siap disimpan ke Git dengan git commit.

5.2 GIT Commit

```
PS D:\praktikum-apl\post-test> git commit -m "posttest5"
```

Gambar 5.2 git commit itu kayak **menyimpan catatan resmi** dari perubahan yang sudah kamu siapkan dengan git add.

5.3 GIT Push

```
PS D:\praktikum-apl\post-test> git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 689.54 KiB | 7.75 MiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/israndomirhjah/praktikum-apl.git
 9ad1014..e32ed67  main -> main
```

Gambar 5.3 git push itu dipakai untuk **mengirim commit yang ada di komputer lokal ke server Git (misalnya GitHub, GitLab, atau Bitbucket)**. Jadi perubahan yang sudah kamu simpan dengan git commit akan dikirim ke repository online supaya bisa dilihat atau dipakai orang lain.